Testing

Thanks to Starlette, testing **FastAPI** applications is easy and enjoyable.

It is based on HTTPX, which in turn is designed based on Requests, so it's very familiar and intuitive.

With it, you can use pytest directly with **FastAPI**.

Using `TestClient`

Note

To use `TestClient`, first install `httpx`.

Add it to your project:

```
$ uv add httpx
```

Import `TestClient`.

Create a `TestClient` by passing your **FastAPI** application to it.

Create functions with a name that starts with `test_` (this is a standard `pytest` convention).

Use the `TestClient` object the same way as you do with `httpx`.

Write simple `assert` statements with the standard Python expressions that you need to check (again, standard `pytest`).

Python 3.10+

```
from fastapi import FastAPI
from fastapi.testclient import TestClient

app = FastAPI()


@app.get("/")
async def read_main():
```

```
    return {"msg": "Hello World"}


client = TestClient(app)


def test_read_main():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"msg": "Hello World"}
```

Tip

Notice that the testing functions are normal `def`, not `async def`.

And the calls to the client are also normal calls, not using `await`.

This allows you to use `pytest` directly without complications.

Technical Details

You could also use `from starlette.testclient import TestClient`.

FastAPI provides the same `starlette.testclient` as `fastapi.testclient` just as a convenience for you, the developer. But it comes directly from Starlette.

Tip

If you want to call `async` functions in your tests apart from sending requests to your FastAPI application (e.g. asynchronous database functions), have a look at the Async Tests in the advanced tutorial.

Separating tests

In a real application, you probably would have your tests in a different file.

And your **FastAPI** application might also be composed of several files/modules, etc.

FastAPI app file

Let's say you have a file structure as described in Bigger Applications:

```
.
├── app
```

```
│   ├── __init__.py
│   └── main.py
```

In the file `main.py` you have your **FastAPI** app:

Python 3.10+

```Python
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
async def read_main():
    return {"msg": "Hello World"}
```

Testing file

Then you could have a file `test_main.py` with your tests. It could live on the same Python package (the same directory with a `__init__.py` file):

```
.
├── app
│   ├── __init__.py
│   ├── main.py
│   └── test_main.py
```

Because this file is in the same package, you can use relative imports to import the object `app` from the `main` module (`main.py`):

Python 3.10+

```Python
from fastapi.testclient import TestClient

from .main import app

client = TestClient(app)


def test_read_main():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"msg": "Hello World"}
```

...and have the code for the tests just like before.

Testing: extended example

Now let's extend this example and add more details to see how to test different parts.

Extended **FastAPI** app file

Let's continue with the same file structure as before:

```
.
├── app
│   ├── __init__.py
│   ├── main.py
│   └── test_main.py
```

Let's say that now the file `main.py` with your **FastAPI** app has some other **path operations**.

It has a `GET` operation that could return an error.

It has a `POST` operation that could return several errors.

Both *path operations* require an `X-Token` header.

Python 3.10+

```
from typing import Annotated

from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel

fake_secret_token = "coneofsilence"

fake_db = {
    "foo": {"id": "foo", "title": "Foo", "description": "There goes my
hero"},
    "bar": {"id": "bar", "title": "Bar", "description": "The bartenders"},
}

app = FastAPI()


class Item(BaseModel):
    id: str
    title: str
    description: str | None = None


@app.get("/items/{item_id}", response_model=Item)
async def read_main(item_id: str, x_token: Annotated[str, Header()]):
    if x_token != fake_secret_token:
        raise HTTPException(status_code=400, detail="Invalid X-Token
header")
    if item_id not in fake_db:
        raise HTTPException(status_code=404, detail="Item not found")
    return fake_db[item_id]
```

```python
@app.post("/items/")
async def create_item(item: Item, x_token: Annotated[str, Header()]) ->
Item:
    if x_token != fake_secret_token:
        raise HTTPException(status_code=400, detail="Invalid X-Token
header")
    if item.id in fake_db:
        raise HTTPException(status_code=409, detail="Item already exists")
    fake_db[item.id] = item.model_dump()
    return item
```

🤓 Other versions and variants

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel

fake_secret_token = "coneofsilence"

fake_db = {
    "foo": {"id": "foo", "title": "Foo", "description": "There goes my
hero"},
    "bar": {"id": "bar", "title": "Bar", "description": "The bartenders"},
}

app = FastAPI()


class Item(BaseModel):
    id: str
    title: str
    description: str | None = None


@app.get("/items/{item_id}", response_model=Item)
async def read_main(item_id: str, x_token: str = Header()):
    if x_token != fake_secret_token:
        raise HTTPException(status_code=400, detail="Invalid X-Token header")
    if item_id not in fake_db:
        raise HTTPException(status_code=404, detail="Item not found")
    return fake_db[item_id]


@app.post("/items/")
async def create_item(item: Item, x_token: str = Header()) -> Item:
    if x_token != fake_secret_token:
        raise HTTPException(status_code=400, detail="Invalid X-Token header")
    if item.id in fake_db:
        raise HTTPException(status_code=409, detail="Item already exists")
    fake_db[item.id] = item.model_dump()
    return item
```

Extended testing file

You could then update `test_main.py` with the extended tests:

Python 3.10+

```
from fastapi.testclient import TestClient

from .main import app

client = TestClient(app)


def test_read_item():
    response = client.get("/items/foo", headers={"X-Token":
"coneofsilence"})
    assert response.status_code == 200
    assert response.json() == {
        "id": "foo",
        "title": "Foo",
        "description": "There goes my hero",
    }


def test_read_item_bad_token():
    response = client.get("/items/foo", headers={"X-Token": "hailhydra"})
    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Token header"}


def test_read_nonexistent_item():
    response = client.get("/items/baz", headers={"X-Token":
"coneofsilence"})
    assert response.status_code == 404
    assert response.json() == {"detail": "Item not found"}


def test_create_item():
    response = client.post(
        "/items/",
        headers={"X-Token": "coneofsilence"},
        json={"id": "foobar", "title": "Foo Bar", "description": "The Foo
Barters"},
    )
    assert response.status_code == 200
    assert response.json() == {
        "id": "foobar",
        "title": "Foo Bar",
        "description": "The Foo Barters",
    }


def test_create_item_bad_token():
    response = client.post(
        "/items/",
        headers={"X-Token": "hailhydra"},
        json={"id": "bazz", "title": "Bazz", "description": "Drop the
bazz"},
    )
    assert response.status_code == 400
```

```
    assert response.json() == {"detail": "Invalid X-Token header"}


def test_create_existing_item():
    response = client.post(
        "/items/",
        headers={"X-Token": "coneofsilence"},
        json={
            "id": "foo",
            "title": "The Foo ID Stealers",
            "description": "There goes my stealer",
        },
    )
    assert response.status_code == 409
    assert response.json() == {"detail": "Item already exists"}
```

🤓 Other versions and variants

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```
from fastapi.testclient import TestClient

from .main import app

client = TestClient(app)


def test_read_item():
    response = client.get("/items/foo", headers={"X-Token": "coneofsilence"})
    assert response.status_code == 200
    assert response.json() == {
        "id": "foo",
        "title": "Foo",
        "description": "There goes my hero",
    }


def test_read_item_bad_token():
    response = client.get("/items/foo", headers={"X-Token": "hailhydra"})
    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Token header"}


def test_read_nonexistent_item():
    response = client.get("/items/baz", headers={"X-Token": "coneofsilence"})
    assert response.status_code == 404
    assert response.json() == {"detail": "Item not found"}


def test_create_item():
    response = client.post(
        "/items/",
        headers={"X-Token": "coneofsilence"},
        json={"id": "foobar", "title": "Foo Bar", "description": "The Foo
Barters"},
    )
    assert response.status_code == 200
    assert response.json() == {
        "id": "foobar",
        "title": "Foo Bar",
        "description": "The Foo Barters",
    }


def test_create_item_bad_token():
    response = client.post(
        "/items/",
```

```
        headers={"X-Token": "hailhydra"},
        json={"id": "bazz", "title": "Bazz", "description": "Drop the bazz"},
    )
    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Token header"}


def test_create_existing_item():
    response = client.post(
        "/items/",
        headers={"X-Token": "coneofsilence"},
        json={
            "id": "foo",
            "title": "The Foo ID Stealers",
            "description": "There goes my stealer",
        },
    )
    assert response.status_code == 409
    assert response.json() == {"detail": "Item already exists"}
```

Whenever you need the client to pass information in the request and you don't know how to, you can search (Google) how to do it in `httpx`, or even how to do it with `requests`, as HTTPX's design is based on Requests' design.

Then you just do the same in your tests.

E.g.:

- To pass a *path* or *query* parameter, add it to the URL itself.
- To pass a JSON body, pass a Python object (e.g. a `dict`) to the parameter `json`.
- If you need to send *Form Data* instead of JSON, use the `data` parameter instead.
- To pass *headers*, use a `dict` in the `headers` parameter.
- For *cookies*, a `dict` in the `cookies` parameter.

For more information about how to pass data to the backend (using `httpx` or the `TestClient`) check the HTTPX documentation.

Note

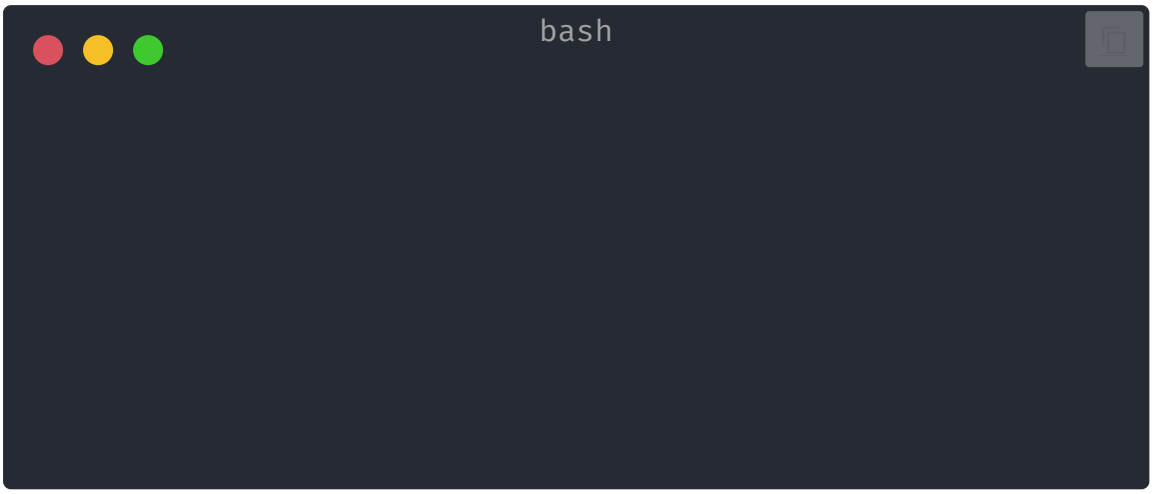
Note that the `TestClient` receives data that can be converted to JSON, not Pydantic models.

If you have a Pydantic model in your test and you want to send its data to the application during testing, you can use the `jsonable_encoder` described in JSON Compatible Encoder.

Run it

After that, you just need to install `pytest`.

Add it to your project:



It will detect the files and tests automatically, execute them, and report the results back to you.

Run the tests with: